E–PIC 代码结构重组与多物理模块化方案（mesh.dict/geometry.dict 统一驱动）

方案整理：ChatGPT（基于 E-PIC-Task3 代码快照）

2026-01-21

摘要

本文给出一份面向 E–PIC 代码的“结构重组 + 可拓展多物理模块化 + 便捷算例管理”的工程方案。核心目标是：**(1) 网格与几何统一由 `mesh.dict` 与 `geometry.dict` 驱动；(2) 物理功能以可插拔 `Module` 形式组织，`main()` 只保留最小启动逻辑；(3) 算例以子目录形式管理，运行时切换 case 而非覆盖根目录 `system`**。方案在尽量少破坏现有代码的前提下，提出分阶段迁移路径、目录结构、字典组织、运行流程与扩展建议。

目录

1 背景与需求梳理 **1**

2 现状快照（基于 E-PIC-Task3 代码结构） **2**

3 目标架构：Core + Geometry/Mesh + Modules + Case **2**

3.1 总体分层 . . . 2

3.2 关键原则 . . . 3

4 建议的目录结构重组 **3**

4.1 建议的目标目录树 . . . 3

4.2 Physics/ 的处理 . . . 4

5 Case 组织与字典定位（解决覆盖 system 的痛点） **4**

5.1 Case 目录约定 . . . 4

5.2 新增 Runtime::Case（字典路径服务） . . . 4

5.3 命令行接口（推荐） . . . 4

5.4 对现有 Dictionary* 的最小侵入改造 . . . 5

6 Runtime 与 Modules：从“单模块入口”走向“多物理可组合” **5**

6.1 模块注册与工厂（避免 if-else 链） . . . 5

6.2 SimContext：共享网格/几何/输出/随机数/并行环境 . . . 5

6.3 模块生命周期建议（为未来多物理耦合预留） . . . 6

7 电喷雾模块化落地：从 Main.cpp 迁移 **6**
7.1 目标：把“电喷雾程序”变成 ElectrosprayModule 6
7.2 Main.cpp 中应迁移的典型内容 6
7.3 ElectrosprayModule 与子求解器的关系 6

8 Run.dict 的建议扩展（支持 case 切换与多模块） **7**

9 迁移实施路线（推荐分 4 个阶段） **7**
9.1 Phase 0：不破坏旧功能的前提下引入 case 7
9.2 Phase 1：ImmersedGeometry 迁移到 Geometry 8
9.3 Phase 2：Physics 拆解并形成 Electricity/Hydro 模块 8
9.4 Phase 3：电喷雾模块落地，main 变薄 8

10 除了你提出的点以外，额外的可拓展/易用性建议 **8**
10.1 建议 1：统一输出与目录管理（OutputManager） 8
10.2 建议 2：字典 include 与默认配置（减少重复） 8
10.3 建议 3：模块间依赖声明与运行管线 9
10.4 建议 4：回归测试与最小算例集 9
10.5 建议 5：明确命名与 API 边界 9

11 风险与对策 **9**

12 附录：建议的最小字典模板 **10**

1 背景与需求梳理

用户需求（归纳）：

1) 以统一的几何/网格定义（`geometry.dict`, `mesh.dict`）作为所有物理模块的基础输入；

2) 将 Electricity、Hydrodynamics、ExplicitPIC 等作为可独立拓展的**功能模块**（Module），未来可继续增加新的物理/数值模块；

3) `main()` 文件“写薄”：把当前主程序里与电喷雾相关的具体流程抽到**电喷雾模块**，像 ExplicitPICModule 一样通过 `RunController` 入口切换；

4) 删除 `Physics/` 目录：其中内容合并到 `Modules/`；`ImmersedGeometry` 移入 `Geometry/` 作为基础能力；

5) 算例管理：不再每次覆盖根目录 `system/`；希望每个算例一个子目录，运行时切换（类似 OpenFOAM 的 case 组织）。

本文方案额外强调：**保持“核心库 + 模块插件 + case”三层解耦**，使得后续增加模块、替换求解器或做多物理耦合时，最少改动、不互相污染。

2 现状快照（基于 E-PIC-Task3 代码结构）

当前目录结构（简化）：

```
include/
  Base/ Discrete/ Field/ Geometry/ IO/ Lagrangian/ Mesh/
  Modules/ExplicitPIC/...
  Physics/ Electricity.h Hydrodynamics.h ImmersedGeometry.h ...
  Runtime/ IModule.h RunController.h
src/
  ...
  Main.cpp
  Physics/ Electricity.cpp Hydrodynamics.cpp
  Runtime/ RunController.cpp
system/
  mesh.dict geometry.dict run.dict pic.dict ...
```

当前已具备的“模块化雏形”：

- `Runtime/IModule`：轻量模块接口；
- `Runtime/RunController`：读取 `./system/run.dict`，目前只识别 `ExplicitPIC`，否则进入 legacy 的巨大 `Main.cpp`；
- `DictionaryRun/DictionaryPIC` 已支持按路径读取；但 `DictionaryMesh/DictionaryGeometry` 仍固定读取 `./system/*.dict`。

主要痛点：

1) **主程序过厚**：电喷雾/电流体等具体流程与底层库混在 `Main.cpp`；
2) **目录语义不清**：`Physics/` 同时承载“可复用的物理场求解器类”和“某些模块化入口”的潜在角色；
3) **字典路径硬编码**：Mesh/Geometry 的字典只能从根目录 `system/` 读，不支持按 case 切换；
4) **运行组织能力弱**：无法声明“多模块耦合/顺序执行/共享网格几何/共享输出”等。

3 目标架构：Core + Geometry/Mesh + Modules + Case

3.1 总体分层

- **Core（基础库）**：Base/IO/Field/Discrete/Lagrangian/Mesh 等，提供通用数据结构与数值工具。
- **Geometry/Mesh 基础能力**：几何描述、水平集、EB/浸入边界几何（`ImmersedGeometry`）等，不与具体物理强耦合。
- **Modules（功能模块）**：每个模块是一个可独立运行的入口（实现 `IModule`），内部可复用 Core 的类与算法。

- **Cases (算例)**: 每个算例一个目录, 包含 system/ 子目录, 存放 mesh.dict, geometry.dict, run.dict 及各模块字典。

3.2 关键原则

1) **网格/几何单一来源**: mesh.dict 与 geometry.dict 由 Runtime 在启动时统一读取并构建 Mesh/Geometry; 模块不得各自重复读一套 "私有网格几何"。

2) **模块配置隔离**: 每个模块只读取自己的 dict (例如 electricity.dict、hydrodynamics.dict、pic.dict、electrospray.dict), 避免 "全局巨 dict"。

3) **main 只做启动**: 解析命令行/确定 case 路径/初始化 RunController/选择模块并运行。

4) **模块可组合**: 电喷雾模块应作为 driver (驱动器模块) 去组合 Electricity/Hydrodynamics/ExplicitPIC 等能力。

4 建议的目录结构重组

4.1 建议的目标目录树

```
E-PIC/
  CMakeLists.txt
  apps/
    epic_main.cpp # 极薄 main
  include/
    Core/... # Base/IO/Field/Discrete/Lagrangian/Mesh
    Geometry/
      LevelSet.* Line.* Phase.*
      ImmersedGeometry.* # 从 Physics 迁入
    Runtime/
      IModule.h
      RunController.*
      Case.h Case.cpp # case 路径/字典定位
      ModuleRegistry.* # 模块注册/工厂
      SimContext.* # 共享上下文 (Mesh/Geometry/输出等)
    Modules/
      ExplicitPIC/
      Electricity/
      Hydrodynamics/
      Electrospray/ # 新: 把 Main.cpp 的电喷雾流程迁入
  src/
    ... 与 include 对应实现 ...
  cases/
    spray2D/
      system/
        run.dict mesh.dict geometry.dict
        electricity.dict hydrodynamics.dict pic.dict electrospray.dict
```

```
constant/（可选）
0/（可选）
```

4.2 Physics/ 的处理

- 将 include/Physics/Electricity.* 迁入 include/Modules/Electricity/ 或 include/Models/Electri
- 将 Hydrodynamics.* 同理迁入。
- ImmersedGeometry.* 迁入 include/Geometry/（建议保持类名不变以减少改动）。
- Physics/ 目录删除。

关于“Electricity/Hydrodynamics 是模块还是可复用 solver 类”的建议：

- 保留它们为**可复用的 solver/model 类**(例如 ElectricitySolver),同时在 Modules/Electricity 下提供一个轻量入口模块 ElectricityModule。
- 这样既满足“模块化运行”，又避免把求解器类写成只能单独跑的脚本式程序。

5 Case 组织与字典定位（解决覆盖 system 的痛点）

5.1 Case 目录约定

建议采用 OpenFOAM 风格最小集合：

- <case>/system/：运行与离散相关字典（网格/几何/模块参数）；
- <case>/output/：统一输出；
- 未来可扩展：constant/（物性/常数）、0/（初场）。

5.2 新增 Runtime::Case（字典路径服务）

新增一个轻量对象 Case：

```
struct Case {
  std::filesystem::path root; // case 根目录
  std::filesystem::path systemDir(); // root/system
  std::filesystem::path dict(std::string name);
};
// 用法：case.dict("mesh.dict") -> <case>/system/mesh.dict
```

5.3 命令行接口（推荐）

建议支持：

```
# 默认：--case ./cases/default
./epic --case ./cases/spray2D

# 允许覆盖 run.dict 的 module
./epic --case ./cases/spray2D --module Electrospray
```

5.4 对现有 Dictionary* 的最小侵入改造

- DictionaryMesh::readDict() 改为 readDict(path) 或构造时传入 Case；
- DictionaryGeometry 同理；
- DictionaryRun/DictionaryPIC 已支持传路径，可直接复用；
- 保留默认路径以兼容旧用法：若未传 case，则仍读 ./system/*.dict。

6 Runtime 与 Modules：从“单模块入口”走向“多物理可组合”

6.1 模块注册与工厂（避免 if-else 链）

当前 RunController 用 if-else 选择模块，模块多了会膨胀。建议引入 ModuleRegistry：

```
using ModuleFactory = std::function<std::unique_ptr<IModule>(const RunConfig&, SimContext
    &)>;
class ModuleRegistry {
public:
  static ModuleRegistry& instance();
  void add(std::string name, ModuleFactory f);
  std::unique_ptr<IModule> create(std::string name, const RunConfig&, SimContext&);
};
// 每个模块 cpp 内 static 注册：REGISTER_MODULE("ExplicitPIC", ExplicitPICModule);
```

6.2 SimContext：共享网格/几何/输出/随机数/并行环境

为满足“网格几何统一构建一次，模块共享”，建议在 Runtime 层建立 SimContext：

```
struct SimContext {
  Case theCase;
  std::unique_ptr<QuadtreeMesh> mesh;
  std::unique_ptr<ImmersedGeometry<QuadtreeMesh>> iGeom;
  OutputManager out; // 统一输出（目录、命名、步号）
  Logger log; // 统一日志
  // 可选：random、MPI/OpenMP 信息、profiling、global parameters...
};
```

构建流程：

1) RunController 解析 case；

2) 读取 `mesh.dict` 构建 mesh；读取 `geometry.dict` 构建 level set / lines / phases；

3) 构建 `ImmersedGeometry(mesh, geometry)`；

4) 再创建并运行选定 Module（Module 仅从 context 拿 mesh/iGeom）。

6.3 模块生命周期建议（为未来多物理耦合预留）

当前 `IModule::run()` 够用，但为了后续“耦合/分裂/多步驱动”，建议逐步扩展为：

```
class IModule {
public:
  virtual void initialize() {}
  virtual void step(int n, double t, double dt) = 0;
  virtual void finalize() {}
};
```

说明：不必一次改完；可以先在 `ElectrosprayModule` 内部实现自己的 step loop，并保持其他模块仍是 run()。

7 电喷雾模块化落地：从 Main.cpp 迁移

7.1 目标：把“电喷雾程序”变成 ElectrosprayModule

建议新建：`Modules/Electrospray/ElectrosprayModule.*`

- **输入字典**：`electrospray.dict`（耦合策略、宏步长、粒子注入、喷射模型参数等）；
- **依赖服务**：从 `SimContext` 获取 mesh/iGeom；内部组合 Electricity/Hydrodynamics/ExplicitPIC 的求解器对象。
- **输出**：统一走 OutputManager，避免各处硬编码 `./output`。

7.2 Main.cpp 中应迁移的典型内容

从当前 `Main.cpp` 可见：时间步计算、输出时间序列、体积分数/界面体积统计、以及电喷雾耦合流程。迁移建议：

1) 把电喷雾专属函数（如 `compute_dt`, `outputTemporalResults`）移入 ElectrosprayModule 内部（或其 `detail` 命名空间）；

2) 把所有硬编码输出路径改成 `context.out.path("time_results.dat")` 之类；

3) 把模型常数（`character_rho` 等）从代码常量迁移到 `electrospray.dict` 或 `constant/`。

7.3 ElectrosprayModule 与子求解器的关系

推荐把 Electricity/Hydrodynamics/PIC 当作**可复用求解器类**：

- `ElectricitySolver`：提供 assemble/solve/gather 等；

- `HydrodynamicsSolver`：提供压力速度耦合、界面重构等；
- `ExplicitPICDriver`：显式 PIC 的推进与沉积。

ElectrosprayModule 作为 driver：

```
initialize():
  build fields, read dicts
for n=0..N-1:
  (1) hydrodynamics predictor/corrector
  (2) electricity solve (with updated interface)
  (3) PIC advance / injection / collisions
  (4) diagnostics + output
finalize()
```

这样既满足“电喷雾是一个入口模块”，又保留底层求解器可被其他 driver 复用（例如“等离子体放电模块”“介质界面模块”等）。

8 Run.dict 的建议扩展（支持 case 切换与多模块）

保持现有风格，建议在 `run.dict` 支持：

```
Run
{
  module Electrospray
  # 可选：case 由命令行指定；也可在此声明
  # case ./cases/spray2D
  logLevel info
}

Modules
{
  Electrospray { dict electrospray.dict }
  Electricity { dict electricity.dict }
  Hydro { dict hydrodynamics.dict }
  PIC { dict pic.dict }
}
```

说明：`Modules` 段仅用于声明“每个模块的配置文件名”，避免把所有参数都写进 run.dict。

9 迁移实施路线（推荐分 4 个阶段）

9.1 Phase 0：不破坏旧功能的前提下引入 case

1) 新增 `Runtime::Case`；`RunController` 增加命令行解析（`--case`, `--module`）。

2) 改造 `DictionaryMesh/DictionaryGeometry` 支持按路径读取。

3) 若未提供 case：默认仍用 `./system`，确保旧用法不崩。

9.2 Phase 1：ImmersedGeometry 迁移到 Geometry

1) 移动头文件与实现到 `Geometry/`；

2) 修正 include 路径与 CMake；

3) 对外接口保持不变（类名/函数签名尽量不动）。

9.3 Phase 2：Physics 拆解并形成 Electricity/Hydro 模块

1) 把 `Electricity.* / Hydrodynamics.*` 迁入 `Modules/`（或 `Models/` + `Modules/` 双层）；

2) 增加 `ElectricityModule` 与 `HydrodynamicsModule` 的最小入口（可先只做 dry-run/单步求解/输出验证）；

3) RunController 通过 ModuleRegistry 支持多模块选择。

9.4 Phase 3：电喷雾模块落地，main 变薄

1) 新建 `ElectrosprayModule`，把 `Main.cpp` 里电喷雾相关流程迁入；

2) `apps/epic_main.cpp` 只保留：初始化、选择 case、创建 context、run 模块；

3) legacy Main.cpp 可暂时保留为 `LegacyElectrospray` 模块或直接删除（建议先保留一段时间便于回归对比）。

10 除了你提出的点以外，额外的可拓展/易用性建议

10.1 建议 1：统一输出与目录管理（OutputManager）

现有代码中输出路径多处硬编码 `./output`。建议新增 `OutputManager`：

- 所有输出都写到 `<case>/output/`；
- 支持按步号自动建子目录（`output/000100/`），避免文件互相覆盖；
- 为后续并行（MPI rank）输出预留前缀。

10.2 建议 2：字典 include 与默认配置（减少重复）

很多 case 之间只有少量参数差异。建议字典支持：

- `include "../defaults/mesh.dict"` 之类；
- 变量替换（例如 `dt = ${dt_macro}`）。

若暂时不想增强解析器，也可通过“默认模板复制 + override 小文件”的方式实现。

10.3 建议 3：模块间依赖声明与运行管线

多物理 driver 之外，有时希望顺序运行多个模块（例如先生成几何，再求电，再跑 PIC）。建议 run.dict 支持：

```
Run { pipeline ( GeometryCheck Electricity PIC ) }
```

并由 RunController 循环创建并运行。

10.4 建议 4：回归测试与最小算例集

重构期最怕“改完能编译但结果悄悄变了”。建议建立：

- **unit tests**：字典解析、网格生成、EB 几何度量等；
- **mini cases**：Poisson/Laplace、静态界面几何、PIC charge deposition 守恒等；
- 在 CMake 里加入 `ctest`，每次改动自动跑。

10.5 建议 5：明确命名与 API 边界

建议约定：

- `*Solver`：可复用求解器类（不含主循环）；
- `*Module`：可独立运行入口（含主循环或 driver）；
- `Dictionary*`：只做配置解析，不承载业务逻辑。

11 风险与对策

风险	典型表现	对策
重构期间功能回归	编译过但结果变、输出丢失	Phase 0 起就建立 mini case + 输出对比；保留 legacy 模式一段时间
字典体系膨胀	参数分散、难以定位	约定每模块一个 dict；run.dict 只做选择与引用；提供 defaults/include
模块间共享数据混乱	字段重复、生命周期不清	SimContext 统一托管 mesh/geometry/output；模块只拿引用
未来并行/性能改动困难	日志/输出/随机数各自为政	统一 OutputManager/Logger/Random；预留 MPI/OpenMP 信息

12 附录：建议的最小字典模板

A. run.dict

```
Run
{
  module Electrospray
  logLevel info
}

Modules
{
  Electrospray { dict electrospray.dict }
  Electricity { dict electricity.dict }
  Hydro { dict hydrodynamics.dict }
  PIC { dict pic.dict }
}
```

B. mesh.dict / geometry.dict

保持你现有格式不变，只把文件位置挪到 `<case>/system/`。

C. electrospray.dict（示意）

```
Electrospray
{
  dtMacro 1e-8
  steps 10000
  coupling semiImplicit
  outputEvery 50
  # model constants / scaling...
}
```